

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 1365

**Web-aplikacija za kolaborativno planiranje
putovanja**

Gabriela Oroz

Zagreb, lipanj 2026

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 1365

**Web-aplikacija za kolaborativno planiranje
putovanja**

Gabriela Oroz

Zagreb, lipanj 2026

DIPLOMSKI ZADATAK br. 1365

Pristupnica: **Gabriela Oroz (0036543637)**
Studij: Računarstvo
Profil: Programsko inženjerstvo i informacijski sustavi
Mentor: prof. dr. sc. Igor Mekterović

Zadatak: **Web-aplikacija za kolaborativno planiranje putovanja**

Opis zadatka:

Organizacija zajedničkog putovanja predstavlja složen problem koji uključuje koordinaciju više korisnika, razmjenu informacija u stvarnom vremenu te donošenje zajedničkih odluka. U praksi se za takve potrebe koriste različiti, nepovezani alati (aplikacije za dopisivanje, web preglednici, bilješke), što dovodi do fragmentacije informacija i smanjene učinkovitosti planiranja. Cilj ovog diplomskog zadatka je istražiti i definirati arhitekturu moderne web aplikacije koja omogućuje kolaborativno planiranje putovanja unutar jedinstvene platforme. U sklopu zadatka potrebno je analizirati tehničke izazove sinkronizacije podataka u okruženju s više istovremenih korisnika te usporediti postojeće pristupe rješavanju konkurentnosti, s naglaskom na CRDT-ove (engl. Conflict-free Replicated Data Types). Na temelju analize potrebno je odabrati prikladan pristup i predložiti tehnologije koje omogućuju pouzdanu sinkronizaciju podataka, podršku za rad bez mrežne veze te nisku latenciju komunikacije u stvarnom vremenu. Dodatni cilj zadatka je istražiti mogućnosti integracije velikih jezičnih modela u kontekstu planiranja putovanja. Razmotriti uporabu RAG (engl. Retrieval-Augmented Generation) tehnike kako bi se zaobišla ograničenja velikih jezičnih modela (LLM) vezana uz zastarjele podatke, naveli izvori podataka, smanjile halucinacije, itd. Donijeti ocjenu ostvarenog pristupa, komentirati najveće izazove i eventualna otvorena pitanja u razvoju ovakve vrste aplikacija te dati smjernice za budući razvoj.

Rok za predaju rada: 26. lipnja 2026.

Sažetak

Naslov: Web-aplikacija za kolaborativno planiranje putovanja

Sažetak: Ovaj rad opisuje web aplikaciju za zajedničko planiranje putovanja. Aplikacija omogućuje da više korisnika u isto vrijeme uređuju isti itinerar, pri čemu se izmjene automatski spajaju bez gubitka podataka. Plan je dostupan i bez mrežne veze, a pri uspostavi veze izmjene se automatski sinkroniziraju. Kada korisnik doda mjesto u plan, sustav automatski dohvaća informacije o tom mjestu korištenjem RAG pristupa. Sustav je izgrađen kao monorepo s tri procesa: Next.js frontend, FastAPI backend i Hocuspocus WebSocket server, a podaci se pohranjuju u PostgreSQL bazu kojom upravlja Supabase.

Ključne riječi: zajedničko uređivanje, CRDT, Yjs, offline, RAG, LLM, Next.js, FastAPI

Summary

Title: Web-application for collaborative travel planning

Summary: This paper describes a web application for collaborative travel planning. The application allows multiple users to edit the same itinerary at the same time, with changes automatically merged without data loss. A plan is available offline as well, and when the connection is restored, changes are automatically synced. When a user adds a place to a plan, the system automatically fetches information about that place using a RAG approach. The system is built as a monorepo with three processes: a Next.js frontend, a FastAPI backend and a Hocuspocus WebSocket server, with data stored in a PostgreSQL database managed by Supabase.

Keywords: collaborative editing, CRDT, Yjs, offline, RAG, LLM, Next.js, FastAPI

Sadržaj

Sažetak.....	5
Summary.....	6
Sadržaj	7
Uvod	8
1. Zahtjevi.....	9
1.1. Funkcijski zahtjevi.....	9
1.2. Nefunkcijski zahtjevi.....	10
2. Pregled sličnih rješenja.....	11
2.1. Wanderlog	11
2.2. TripIt.....	12
2.3. Roadtrippers.....	12
2.4. Google Travel.....	13
2.5. Usporedba.....	14
3. Podatkovni model rješenja.....	15
3.1. Relacijska shema	17
3.2. Yjs dokument.....	18
4. Opis rješenja	19
4.1. Korištene tehnologije.....	19
4.2. Arhitektura sustava	20
4.3. Suradnja u stvarnom vremenu	22
4.3.1. Načelo rada CRDT-a	22
4.3.2. Primjena u web aplikaciji Ajmo	23
4.4. Rad bez mrežne veze	26
4.5. AI obogaćivanje sadržaja.....	28
4.5.1. Interakcije s LLM-om.....	29
4.6. Sigurnost.....	32
4.7. Stranice i komponente	33
5. Budući rad.....	39
Zaključak	40
Literatura	41
Izjava o korištenju umjetne inteligencije.....	42

Uvod

Organizacija zajedničkog putovanja predstavlja složen problem koji uključuje koordinaciju više korisnika, razmjenu informacija u stvarnom vremenu te donošenje zajedničkih odluka. U praksi se za takve potrebe koriste različiti, nepovezani alati (aplikacije za dopisivanje, web preglednici i bilješke), što dovodi do fragmentacije informacija i smanjene učinkovitosti planiranja.

Cilj ovog diplomskog rada je istražiti i definirati arhitekturu moderne web aplikacije koja omogućuje kolaborativno planiranje putovanja unutar jedinstvene platforme. U sklopu rada analiziraju se tehnički izazovi sinkronizacije podataka u okruženju s više istovremenih korisnika te uspoređuju postojeći pristupi rješavanju konkurentnosti, s naglaskom na CRDT (engl. *Conflict-free Replicated Data Types*). Također se istražuju mogućnosti integracije velikih jezičnih modela primjenom RAG-a (engl. *Retrieval-Augmented Generation*) kako bi se zaobišla ograničenja velikih jezičnih modela vezana uz zastarjele podatke i smanjile halucinacije.

Rezultat ovog rada je Ajmo, web aplikacija koja u jedinstvenoj platformi spaja zajedničko uređivanje itinerara u stvarnom vremenu, rad bez mrežne veze te AI (engl. *Artificial Intelligence*) obogaćivanje sadržaja.

1. Zahtjevi

Aplikacija je zamišljena kao platforma na kojoj korisnici mogu zajedno planirati putovanja tako što naprave plan, pozovu prijatelje, uređuju itinerar, uz AI obogaćuju sadržaj i služe se platformom čak i bez mrežne veze. Da bi to sve radilo kako treba, sustav mora ispuniti određene funkcijske i nefunkcijske zahtjeve.

1.1. Funkcijski zahtjevi

Registracija i profil - Neregistrirani korisnici mogu vidjeti samo početnu stranicu. Registracija je moguća putem e-maila i lozinke ili Google računa. Nakon registracije korisnik može postaviti ime, profilnu sliku, opis i odabrati preferencije (interesi, prehrana i budžet), koje LLM (engl. *Large Language Model*) uzima u obzir pri generiranju AI prijedloga.

Planovi - Korisnik može napraviti, pregledavati, mijenjati i brisati planove. Svaki plan ima naslov, opis, datume, naslovnu sliku i razinu vidljivosti. Plan može imati više destinacija, gdje je svaka destinacija par država-grad s pridruženim datumima. Na početnoj stranici prijavljeni korisnik vidi svoje planove, planove na koje je pozvan i javne planove drugih korisnika.

Uređivanje itinerara - Sadržaj plana organiziran je po danima, a svaki dan sadrži popis stavki koje mogu biti atrakcije, aktivnosti, restorani, hoteli ili prijevoz. Svaka stavka ima naslov, lokaciju, vrijeme, trajanje i bilješke.

Suradnja - Plan se može dijeliti na tri načina: dodavanje članova s određenom ulogom (čitatelj, uređivač ili vlasnik), putem poveznice ili promjenom razine vidljivosti. Kada više korisnika u isto vrijeme uređuje plan, izmjene su svima vidljive odmah. Ako dvoje korisnika u isto vrijeme piše u isto tekstualno polje, izmjene se automatski spajaju bez gubitka podataka. Sučelje prikazuje koji su korisnici trenutno aktivni i što uređuju.

AI obogaćivanje sadržaja - Kada korisnik unese naziv stavke, sustav automatski dohvaća opis, lokaciju, radno vrijeme, cijenu, sliku i dodatna polja ovisno o tipu stavke (npr. kuhinju za restorane ili sadržaje za hotele). Također, sustav nudi prijedloge na temelju korisnikovih preferencija i onog što već postoji u planu.

Društveni dio - Članovi plana mogu komentirati plan ili svaku stavku zasebno, reagirati na stavke i ocjenjivati ih ocjenama od jedan do pet. Sve izmjene unutar plana vidljive su u pregledu aktivnosti. Također, korisnici mogu slati zahtjeve za prijateljstvo.

Rad bez mrežne veze - Plan koji je korisnik prethodno otvorio dostupan je i bez mrežne veze, a uređivanje je moguće nastaviti lokalno. Pri ponovnoj uspostavi veze izmjene se automatski sinkroniziraju sa serverom i spajaju s promjenama ostalih korisnika bez gubitka podataka i bez duplikata. Sučelje uvijek prikazuje trenutno stanje veze.

1.2. Nefunkcijski zahtjevi

Brzina lokalnih izmjena - Svaka promjena koju korisnik napravi prikazuju se odmah, u otprilike manje od 100 milisekundi. To je moguće jer se izmjene prvo upisuju lokalno u Yjs dokument u pregledniku, a slanje na server se obavlja u pozadini.

Spajanje promjena - Kada više korisnika u isto vrijeme mijenja isti sadržaj, sustav automatski dolazi do konzistentnog stanja bez gubitka podataka. To je osigurano matematičkim svojstvima CRDT-a koje koristi Yjs biblioteka.

Sigurnost promjena bez mrežne veze - Izmjene napravljene bez mrežne veze ostaju sačuvane čak i ako korisnik osvježi stranicu ili mu istekne sesija, a pri uspostavi veze se ispravno spajaju s promjenama ostalih korisnika.

Zaštita ključeva - Supabase service-role ključ i API ključevi AI pružaci usluge dostupni su isključivo na serveru. Sve operacije koje zahtijevaju poseban pristup izvršavaju se na server strani.

Otvoreni geoservisi - Za usmjeravanje i geokodiranje koriste se samo servisi otvorenog koda (OpenStreetMap Nominatim, Photon za pretvaranje naziva mjesta u koordinate, OSRM za rute autom, biciklom i pješke i Transitous za javni prijevoz).

Ograničenje troška LLM-a - Promjenjivi podaci (npr. radno vrijeme ili cijena) dohvaćeni za neko mjesto spremaju se u bazu podataka uz rok trajanja od 24 sata. Dok rok ne istekne, sustav vraća postojeće podatke umjesto da ponovno poziva LLM. Ako više korisnika u isto vrijeme traži obogaćivanje sadržaja za isto mjesto, pokreće se samo jedan poziv i svi čekaju na isti rezultat.

Autorizacija - Sadržaj plana vidljiv je samo njegovim članovima, osim ako vlasnik ne postavi drugačiju razinu vidljivosti. Uređivanje je moguće samo korisnicima s ulogom uređivača ili vlasnika. Korisnici s ulogom čitatelja mogu samo pregledavati sadržaj.

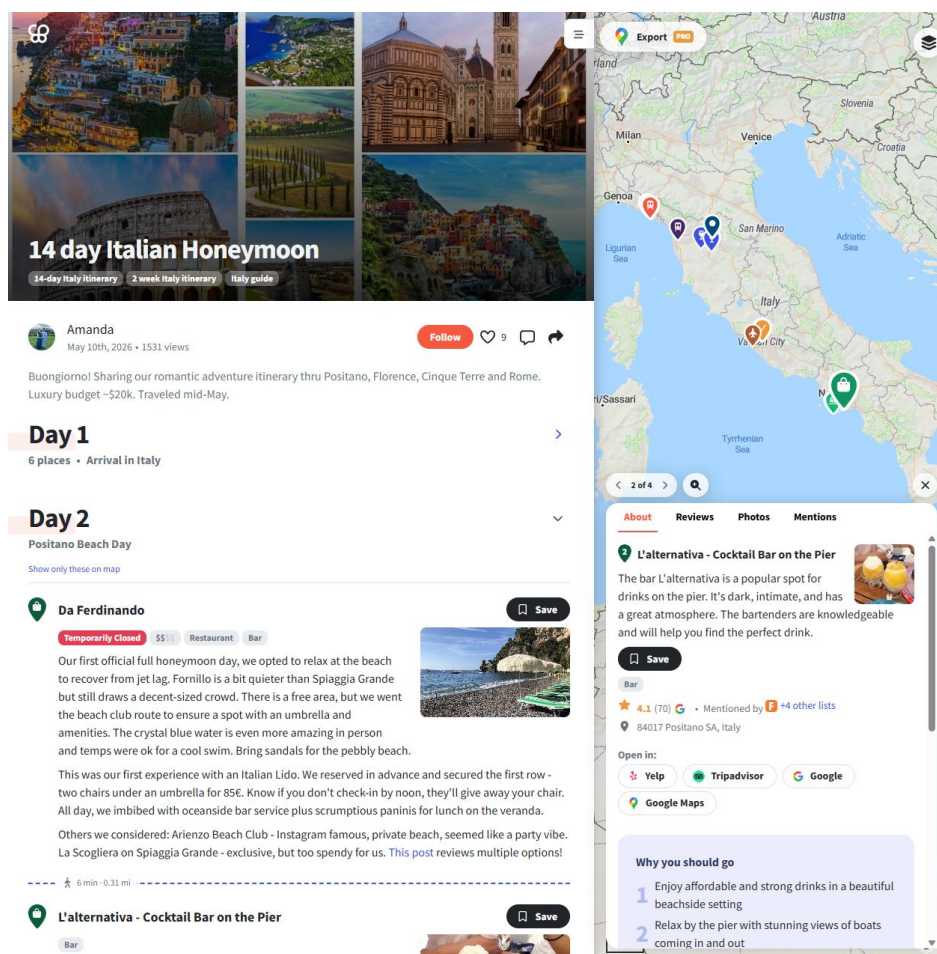
2. Pregled sličnih rješenja

Trenutno postoji nekoliko aplikacija koje pokrivaju dio funkcionalnosti koje nudi Ajmo, ali svaki od njih ima drugačiji fokus i određena ograničenja.

2.1. Wanderlog

Wanderlog, koji je prikazan na Sl. 1, je najbližiji aplikaciji Ajmo. Podržava zajedničko uređivanje, itinerare po danima, pretragu mjesta putem *Google Places*, prikaz ruta na karti te praćenje hotela i letova. Dostupan je i rad bez mrežne veze, ali samo u plaćenju verziji. Podaci o mjestima sadrže ono što vraća *Google Places* (ime, adresa, ocjena i radno vrijeme).

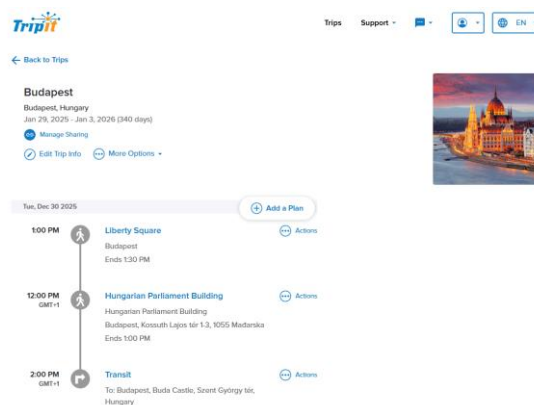
[1]



Sl. 1. Wanderlog

2.2. Triplt

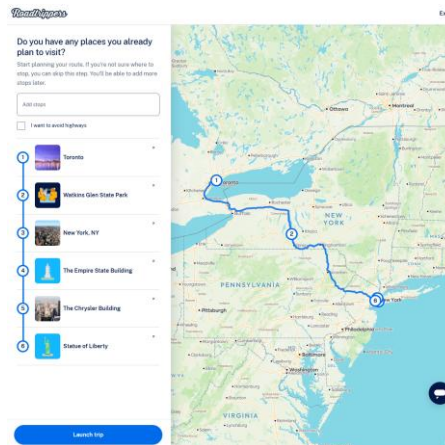
TripIt, koji je prikazan na Sl. 2, pristupa planiranju putovanja na drugačiji način. Fokusiran je na organizaciju već napravljenih rezervacija. Korisnici prosljeđuju potvrde rezervacija na zajedničku e-adresu, a servis ih složi u vremensku traku. Mjesta i atrakcije moguće je dodati ručno, ali informacije (npr. radno vrijeme, slika ili opis) korisnik mora sam unijeti. Plan se može dijeliti s drugim korisnicima, ali samo vlasnik može mijenjati plan. Zajedničko uređivanje plana i AI obogaćivanje sadržaja nisu podržani, a rad bez mrežne veze je ograničen na čitanje. [2]



Sl. 2. TripIt

2.3. Roadtrippers

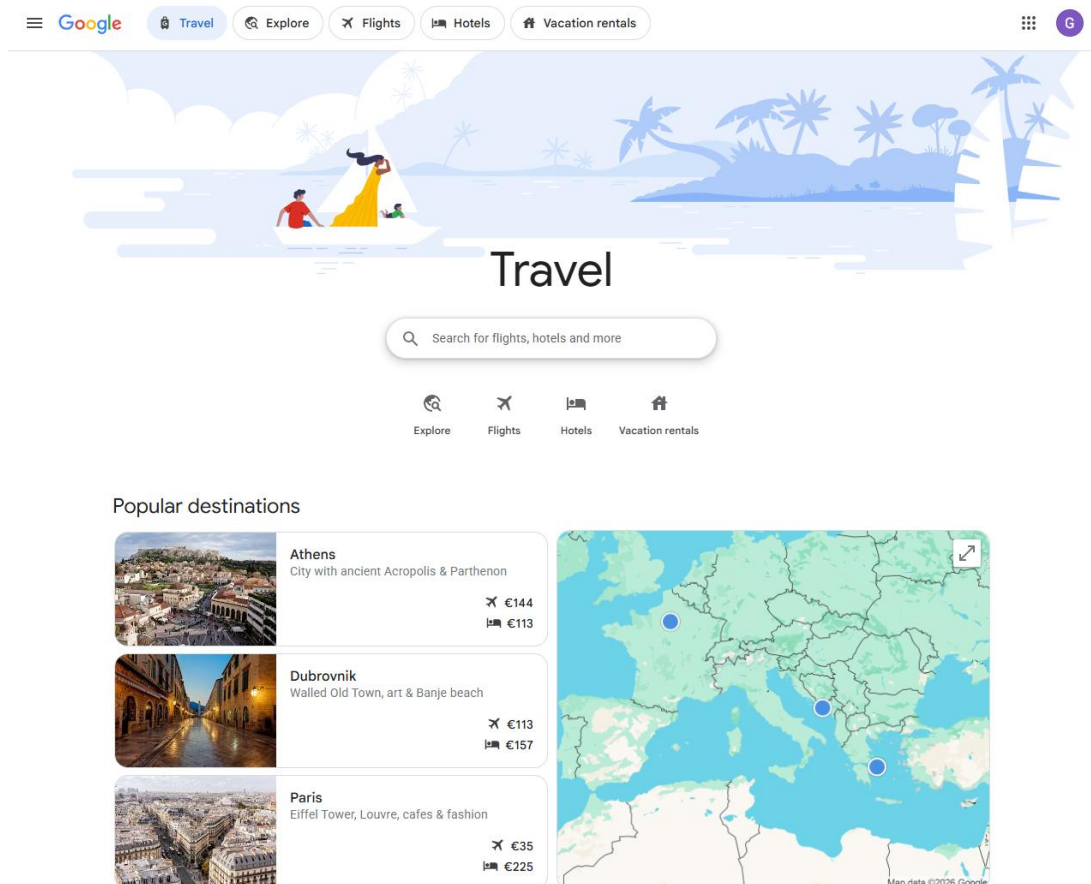
Roadtrippers, koji je prikazan na Sl. 3, je namijenjen putovanjima autom i to uglavnom po Sjevernoj Americi. Ideja je da korisnik odabere mjesta koja želi posjetiti usput (npr. atrakcije i restorane), a aplikacija to složi u rutu i izračuna koliko će vožnja trajati. Plaćena verzija uključuje zajedničko uređivanje, ali aplikacija ne podržava AI obogaćivanja sadržaja i rad bez mrežne veze. [3]



Sl. 3. Roadtrippers

2.4. Google Travel

Google Travel, koji je prikazan na Sl. 4, je aplikacija koja automatski uvozi potvrde rezervacija iz Gmaila i prikazuje ih kao pregled putovanja. Služi uglavnom za pregled letova i smještaja, a ne za planiranje itinerara od nule. Nema zajedničkog uređivanja, AI obogaćivanja sadržaja niti rada bez mrežne veze. [4]



Sl. 4. Google Travel

2.5. Usporedba

Tablica 1. prikazuje usporedbu mogućnosti opisanih aplikacija s onime što nudi Ajmo. Niti jedna postojeća aplikacije ne pokriva sve tri glavne stvari: zajedničko uređivanje u stvarnom vremenu, rad bez mrežne veze i AI obogaćivanje sadržaja. Ta kombinacija čini Ajmo drugačijim od onoga što trenutno postoji.

Mogućnost	Wanderlog	TripIt	Roadtrippers	Google Travel	Ajmo
Uređivanje u stvarnom vremenu	Da	Ne	Ograničeno	Ne	Da
Dnevna struktura itinerara	Da	Da	Ne	Ručno	Da
Hoteli povezani s danima	Da	Da	Ne	Djelomično	Da
Uređivanje izvan mreže	Samo plaćeno	Samo čitanje	Ne	Ne	Da
AI opisi i savjeti	Ne	Ne	Ne	Ne	Da
Prijedlozi međugradskog prijevoza	Ograničeno	Samo rezervacije	Ne	Odvojeni alat	Da
Otvoreni geoservisi	Ne	Ne	Ne	Ne	Da
AI prijedlozi prema preferencijama	Ne	Ne	Ne	Ne	Da

Tablica 1. Usporedba mogućnosti Ajmo s postojećim alatima

3. Podatkovni model rješenja

Podaci aplikacije spremaju se u PostgreSQL bazu podataka kojom upravlja Supabase. Baza sadrži 19 tablica koje se mogu vidjeti na Sl. 5 i Supabase *auth* tablicu. One pokrivaju sve dijelove sustava: korisničke profile i preferencije, strukturu planova i itinerara, društvene interakcije (komentari, oznake svidanja i prijateljstva), pozivnice i poveznice za dijeljenje te podatke koje dohvaća LLM.

Uz relacijske tablice, svaki plan ima i Yjs dokument koji se koristi za zajedničko uređivanje u stvarnom vremenu. Taj dokument se ne sprema odvojeno nego se njegova snimka čuva u bazi kao jedan od stupaca tablice planova. Dokument postoji u memoriji servera i preglednika dok je plan otvoren, a u bazu podataka se zapisuje nakon nekog vremena neaktivnosti i kad svi korisnici zatvore plan. Način rada Yjs dokumenta i sinkronizacija s relacijskim tablicama opisani su u poglavlju 4.3.2.



Sl. 5. Baza podataka

3.1. Relacijska shema

Tablice vezane uz korisnike čuvaju osnovne podatke profila (npr. ime, korisničko ime i profilnu sliku) i preferencije korisnika (interesi, prehrana i budžet).

Jezgru sustava čine tablice vezane uz planove i itinerare. Svaki plan ima naslov, opis, datume, razinu vidljivosti, naslovnu sliku i snimku Yjs dokumenta. Plan se dijeli na destinacije, dane i stavke. Destinacije određuju koje gradove plan pokriva i kojim redoslijedom, dok se dani sastoje od datuma, bilježaka i stavki (npr. atrakcije, restorani i prijevoz). Hoteli su izdvojeni na razini plana jer obično pokrivaju više dana. Stavke imaju *sort key* koji omogućuje da se nova stavka ubaci između dvije postojeće bez da se mijenja redoslijed svega ostalog.

Društveni dio pokriva sve interakcije između korisnika. Svaki plan može imati više članova s dodijeljenom ulogom, a za pozivanje novih članova koriste se *invite* tokeni koji mogu imati ograničen broj korištenja ili rok trajanja. Komentari podržavaju odgovore i *soft delete*, što znači da odgovori ostaju vidljivi čak i nakon što je originalni komentar obrisano. Također, korisnici mogu reagirati na stavke i ocjenjivati ih, a sve izmjene unutar plana bilježe se u pregledu aktivnosti.

Za potrebe LLM-a postoje tablice koje čuvaju podatke o mjestima. Fiksne informacije (naziv, koordinate, opis i slika) spremaju se trajno, a one koje se mijenjaju (radno vrijeme, cijena i savjeti) spremaju se s rokom trajanja od 24 sata. Nakon toga ih pozadinski proces briše, a novim zahtjevom se dohvaćaju ispočetka. Budući da korisnici mogu upisati isto mjesto na više različitih načina, postoji tablica koja svaki unos mapira na jedan kanonski naziv mjesta.

3.2. Yjs dokument

Yjs dokument je CRDT struktura podataka koja omogućuje da istovremene izmjene više korisnika budu spojene u isti rezultat kod svih članova bez ručnog rješavanja konflikata [5]. Svaki plan ima jedan takav dokument, a identificira se po UUID-u plana. Način na koji CRDT spaja istovremene izmjene objašnjen je u poglavlju 4.3.1.

Dokument sadrži šest dijelova: stavke, bilješke po danima, metapodatke plana, oznake sviđanja, ocjene i komentare. Bilješke dana i bilješke stavki nisu obični tekst nego Yjs tekstualni tip, što je posebna struktura koja prati svaki znak zasebno i time omogućuje spajanje izmjena na razini znaka. Metapodaci plana (naslov, opis, datumi i vidljivost plana) trajno se čuvaju u relacijskim tablicama, a u dokumentu se drži samo njihova kopija kako bi izmjene bile odmah vidljive svim članovima. Oznake sviđanja i ocjene vežu se uz pojedinu stavku i korisnika, a komentari podržavaju odgovore. Korijenska struktura dokumenta se može vidjeti na Sl. 6, a pojednostavljeni primjer sadržaja dokumenta jednog plana na Sl. 7.

```
Y.Doc
├── items      Y.Map< day_id → Y.Array<Y.Map> >
├── day_notes  Y.Map< day_id → Y.Text >
├── plan_meta  Y.Map< string | null >
├── likes      Y.Map< item_id → Y.Map<user_id, true> >
├── ratings    Y.Map< item_id → Y.Map<user_id, 1..5> >
└── comments  Y.Array<Y.Map>
```

Sl. 6. Korijenska struktura Yjs dokumenta

```
{
  "items": {
    "day-1": [
      { "id": "item-1", "item_type": "attraction",
        "title": "Muzej prekinutih veza", "notes": "<Y.Text>", "sort_key": "a0" },
      { "id": "item-2", "item_type": "restaurant", "title": "Ručak", "sort_key": "a1" }
    ]
  },
  "day_notes": { "day-1": "<Y.Text> Kupiti karte unaprijed" },
  "plan_meta": { "title": "Vikend u Zagrebu", "visibility": "private" },
  "likes": { "item-1": { "user-gabi": true } },
  "ratings": { "item-1": { "user-gabi": 5 } },
  "comments": [
    { "id": "c-1", "plan_item_id": "item-1", "parent_id": null, "body": "Super!" }
  ]
}
```

Sl. 7. Pojednostavljeni primjer sadržaja Yjs dokumenta

4. Opis rješenja

4.1. Korištene tehnologije

Sustav je izgrađen kao monorepo s tri procesa: Next.js frontend, FastAPI backend i Hocuspocus WebSocket server. Svi dijele istu PostgreSQL bazu podataka kojom upravlja Supabase. Opis glavnih tehnologija koje su korištene:

Next.js

Next.js je framework za React koji se koristi za izgradnju korisničkog sučelja. Koristi App Router, pa se rute, layouti i komponente nalaze u istom direktoriju. Neke komponente se generiraju na serveru i mogu čitati direktno iz baze, a neke izvršavaju u pregledniku i zadužene su za interaktivne dijelove stranice poput uređivača itinerara. *Middleware* presreće svaki zahtjev, verificira JWT (engl. *JSON Web Token*) iz kolačića i korisnike koji nisu prijavljeni preusmjerava na stranicu za prijavu. [6]

React i TypeScript

React je JavaScript biblioteka za izgradnju korisničkih sučelja [7]. Komponente su organizirane po funkcionalnosti, a tipovi se dijele između frontenda i Hocuspocus servera. TypeScript se koristi kako bi se greške uhvatile već prilikom pisanja koda. [8]

Yjs i Hocuspocus

Yjs je biblioteka za CRDT strukture podataka koja omogućuje zajedničko uređivanje u stvarnom vremenu. Izmjene više korisnika se uvijek spajaju u isti rezultat bez obzira kojim redoslijedom su stigle. Ista stvar radi i za izmjene napravljene bez mrežne veze. [5]

Hocuspocus je WebSocket server za Yjs. Ima *hookove* za autentifikaciju, učitavanje i spremanje dokumenta te *awareness* kanal. Korisnici s ulogom čitatelja dobivaju konekciju samo za čitanje pa server automatski odbacuje sve njihove dolazne poruke. [9]

FastAPI

FastAPI je Python web framework koji se koristi za REST krajnje točke, AI *pipeline* i interne krajnje točke [10] koje poziva Hocuspocus server. Python je odabran zbog AI dijela jer LangChain puno bolje funkcionira u Pythonu nego u JavaScriptu, pogotovo za rad s različitim LLM pružateljima usluge.

LangChain

LangChain je Python biblioteka za rad s jezičnim modelima. Koristi se jer ima jedinstveno sučelje prema različitim LLM pružateljima usluge, podržava strukturirani izlaz [11] koji tjera model da vrati odgovor u točno zadanom formatu i ima *streaming* [12] koji se koristi za postupno prikazivanje AI prijedloga. Kada jedan pružatelj usluge dostigne kvotu, aplikacija automatski prelazi na sljedećeg.

Supabase

Supabase daje PostgreSQL bazu, autentifikaciju i spremanje datoteka u jednom paketu. Frontend koristi SDK (engl. *Software Development Kit*) koji poštuje RLS (engl. *Row-Level Security*) pravila, a backend koristi *service role* ključ koji ta pravila zaobilazi jer treba pristup tablicama koje frontend ne smije vidjeti. Supabase Auth radi registraciju, prijavu, oporavak lozinke i Google OAuth. [13] Kad se korisnik registrira, okidač u bazi mu automatski napravi profil.

Vanjski servisi

Za AI *pipeline* koristi se Tavily za web pretragu [14] i Pexels za dohvaćanje slika stavki [15]. Za geokodiranje i usmjeravanje koriste se samo servisi otvorenog koda (Nominatim [16], Photon za pretvaranje naziva mjesta u koordinate [17], OSRM za rute autom, biciklom i pješke i Transitous za javni prijevoz [18]).

4.2. Arhitektura sustava

Ajmo se sastoji od tri procesa koji dijele jednu PostgreSQL bazu, a dijagram arhitekture sustava se može vidjeti na Sl. 8. Korisnik direktno komunicira samo s Next.js frontendom. On prikazuje stranice, u pregledniku drži Yjs dokument i uređivač itinerara, s FastAPI backendom komunicira preko REST-a, a s Hocuspocus serverom preko WebSocketeta. Za jednostavnije upite čitanja koristi Supabase SDK koji ide direktno na bazu.

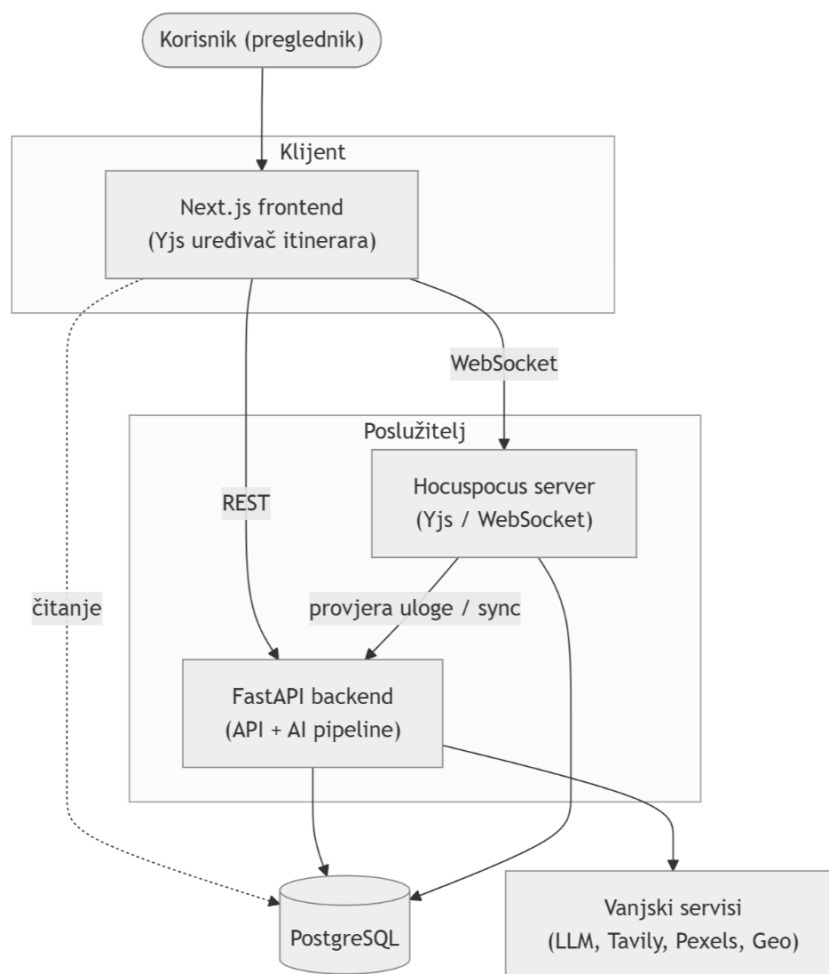
FastAPI server odgovoran je za sve što nije kolaborativno (npr. upravljanje planovima i članovima, prijatelji). Također, pokreće AI *pipeline* i ima krajnje točke koje koristi Hocuspocus server za provjeru uloga i pokretanje sinkronizacije s bazom. Te krajnje točke nisu dostupni izvana i zaštićeni su dijeljenom tajnom u zaglavlju svakog zahtjeva.

Hocuspocus server je Node.js proces koji drži Yjs dokumente u memoriji i jedini je koji obrađuje WebSocket konekcije. Pri svakoj novoj konekciji pita FastAPI backend za ulogu

korisnika u tom planu i ako je korisnik čitatelj, konekcija se označava samo za čitanje. S bazom komunicira direktno kroz PostgreSQL *driver* jer radi sa snimkama Yjs dokumenata, a Supabase SDK ne podržava dobro takav tip podataka. Snimka je kompaktni binarni zapis cijelog stanja dokumenta koji Yjs generira, a u bazi podataka se sprema kao binarni stupac (bytea).

Svi vanjski servisi, uključujući LLM pružaoce usluga, Tavily, Pexels i geoservise, dostupni su samo iz FastAPI backenda. API ključevi nikad ne dolaze do preglednika.

Komunikacija između procesa ide na nekoliko načina. Frontend i FastAPI server komuniciraju preko REST-a. Yjs dokument putuje između frontenda i Hocuspocus server preko WebSoketa. Hocuspocus server i FastAPI server komuniciraju preko HTTPS-a za provjeru uloga i pokretanje sinkronizacije. Za čitanje iz baze frontend koristi Supabase SDK direktno, a sve operacije pisanja idu kroz FastAPI. Oba serverska procesa imaju *service role* ključ, a preglednik ga nikad ne dobiva.



Sl. 8. Arhitektura sustava

4.3. Suradnja u stvarnom vremenu

Za rješavanje istovremenih izmjena više korisnika postoje dva glavna pristupa: operacijska transformacija (engl. *Operational Transformation, OT*) i CRDT (engl. *Conflict-free Replicated Data Type*). Kod OT-a se svaka izmjena šalje kao operacija (npr. „umetni znak na poziciju 5") centralnom poslužitelju koji određuje njihov redoslijed i transformira ih s obzirom na izmjene koje su u međuvremenu stigle od drugih korisnika, a onda ih vraća svim korisnicima tako da svi na kraju imaju isti rezultat. Kod CRDT-a se izmjene ne transformiraju, već je sama struktura podataka napravljena tako da se izmjene uvijek spajaju u isti rezultat bez obzira na redoslijed, pa nije potreban centralni poslužitelj koji ih usklađuje [19]. Za ovaj rad je odabran CRDT jer nije potreban centralni poslužitelj za usklađivanje i podržava rad bez mrežne veze.

4.3.1. Načelo rada CRDT-a

CRDT je tip podataka kod kojeg isti podatak postoji u više kopija, a svaki uređaj ima svoju kopiju. Korisnik radi izmjene nad svojom kopijom i ne mora čekati ostale, pa dvije kopije u nekom trenutku mogu imati različite izmjene. Zadatak CRDT-a je da se te izmjene na kraju spoje u isto stanje bez obzira kojim su redoslijedom stigle i bez ručnog rješavanja konflikata.

CRDT-ovi se dijele na dvije vrste. Pristup zasnovan na stanju (engl. *state-based, CvRDT*) prenosi cijelo stanje koje se zatim spaja kod svih kopija, dok pristup zasnovan na operacijama (engl. *operation-based, CmRDT*) prenosi samo pojedinačne izmjene koje moraju davati isti rezultat bez obzira na redoslijed primjene [20]. Yjs, koji se u ovom radu koristi za sadržaj dokumenta, pripada operacijskoj vrsti jer prenosi samo izmjene koje ostale kopije još nemaju, dok *awareness* kanal opisan u poglavlju 4.3.2 koristi pristup zasnovan na stanju.

Svaka stavka ima jedinstveni identitet oblika (klijent ID, brojač). Klijent ID je nasumičan broj koji označava na kojoj je kopiji stavka nastala, a brojač je logički sat te kopije (broj koji raste sa svakom izmjenom). Taj brojač nema veze sa stvarnim vremenom jer se satovi na različitim uređajima ne mogu pouzdano uskladiti, već služi tome da kopije znaju koje izmjene druge kopije još nisu vidjele i koje su izmjene nastale istovremeno.

Poredak izmjena određuje se prema određenom pravilu. Kad se stavka umeće, zapamti se tko su joj u tom trenutku lijevi i desni susjed kako bi se znalo između koje dvije stavke se

nalazi. Ako dvije kopije u isto vrijeme umetnu stavku na isto mjesto (imaju iste susjede), redosljed se rješava usporedbom klijent ID-a tako da stavka s manjim brojem se postavlja prva. Budući da svaki uređaj uspoređuje iste brojeve po istom pravilu, na kraju imamo isti rezultat na svakom uređaju bez obzira čija je izmjena prije stigla. Brisanje ne miče stavku iz strukture nego je samo označi obrisano. Tako sustav zna gdje staviti nove stavke čak i ako se susjedna stavka u međuvremenu obrisala.

Princip rada CRDT možemo pojasniti jednostavnim primjerom. Ako imamo polje koje sadrži riječ „OKO“ i korisnik A dodaje slovo „S“ prije te riječi i „L“ nakon, a korisnik B u isto vrijeme briše cijelu riječ, nakon spajanja oba korisnika će u polju vidjeti riječ „SL“. Slova „O“, „K“ i „O“ su označena obrisanima, ali i dalje postoje pa sustav zna gdje pripadaju slova „S“ i „L“. Rezultat je isti na svim uređajima, ali izgledom riječ „SL“ nema smisla. To je jer CRDT osigurava da svi korisnici vide jednako, ali ne razumije značenje teksta.

CRDT ima i nekoliko ograničenja. Kad dva korisnika u isto vrijeme pišu na isto mjesto u istom tekstualnom polju, obično se njihovi tekstovi zadrže jedan kraj drugoga, ali ponekad se slova isprepleću. Kod polja koje drže samo jednu vrijednost (npr. naslov), izmjene se rješavaju tako da jedna vrijednost pobijedi, a druga se izgubi bez ikakve obavijesti korisniku.

CRDT radi i kada nestane mrežna veza tako što se izmjene čuvaju lokalno, a spoje se kada se mreža vrati.

4.3.2. Primjena u web aplikaciji Ajmo

Web aplikacija Ajmo za zajedničko uređivanje koristi Yjs biblioteku koja implementira CRDT strukturu opisanu u 4.3.1. poglavlju. Svaki plan ima jedan Yjs dokument, prepoznat po UUID-u plana. Dok je plan otvoren u bilo kojem pregledniku, Yjs dokument je jedini izvor istine za sadržaj itinerara. Sadržaj se također nalazi u snimci u bazi (kako bi se prilikom sljedećeg otvaranja vratilo posljednje stanje) i u relacijskim tablicama (kako bi se plan mogao prikazati i izvan uređivanja, npr. na početnoj stranici). Dokument sadrži šest dijelova: stavke, bilješke po danima, metapodatke plana, oznake sviđanja, ocjene i komentare. Detaljniji opis strukture i primjer Yjs dokumenta nalazi se u 3.2. poglavlju.

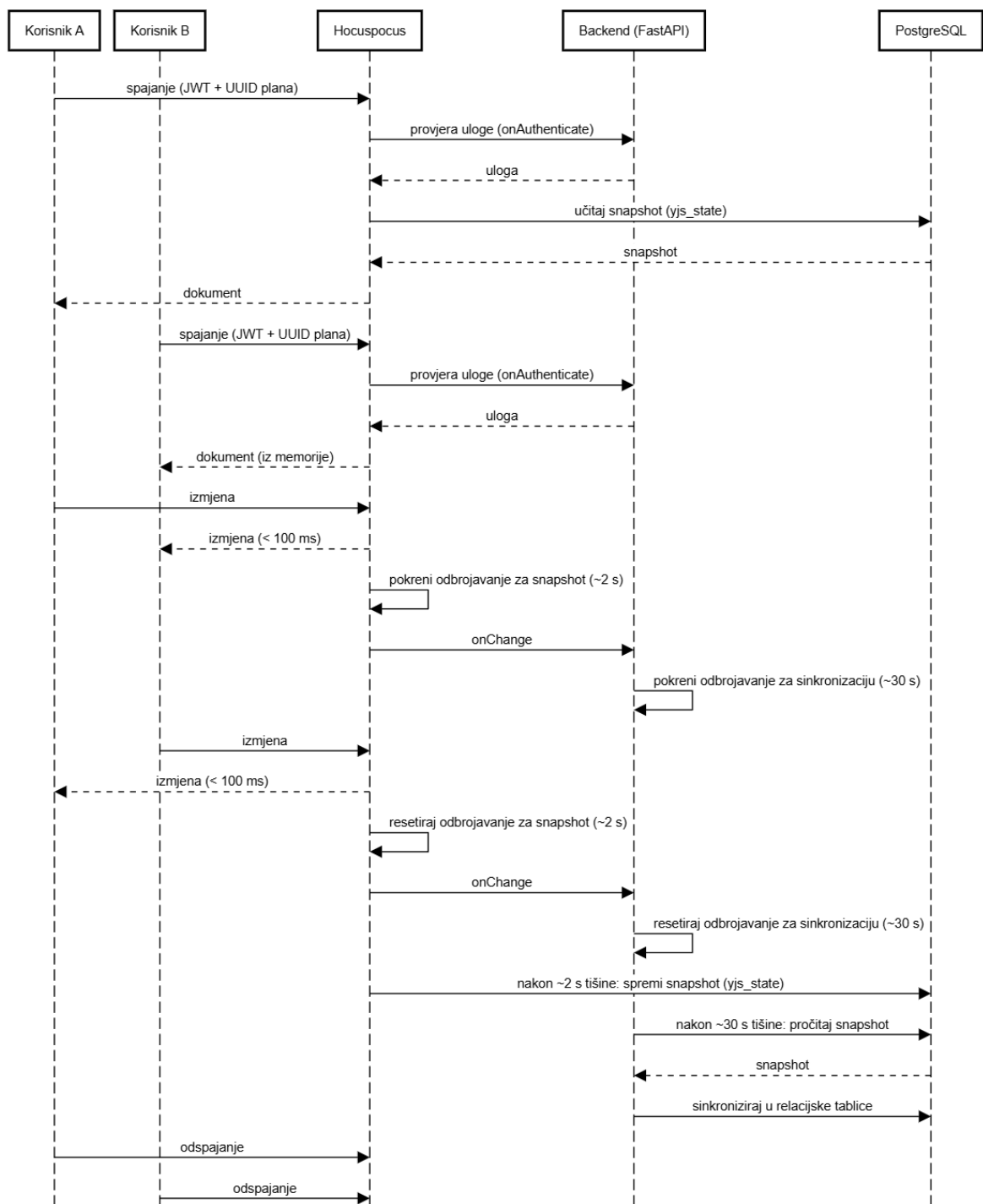
Uz dokument postoji i *awareness* kanal koji prenosi informacije o tome tko je trenutno aktivan i što uređuje, ali se ne sprema u bazu. *Awareness* koristi CRDT koji radi na stanju,

a ne na pojedinačnim izmjenama. Svaki korisnik uz svoje *awareness* stanje drži i brojač. Kad se nešto promijeni, brojač se poveća i cijelo se stanje pošalje ostalim korisnicima. Ostali korisnici usporede primljeni brojač s onim koji već imaju za tog korisnika i zamijene staro stanje novim (ako je novi brojač veći). Ako od nekog korisnika ništa ne stigne 30 sekundi, ostali ga korisnici označavaju kao odsutnog i miču s popisa aktivnih korisnika. Tako se *awareness* razlikuje od sinkronizacije sadržaja dokumenta jer šalje cijelo stanje, a ne samo izmjene koje ostali korisnici još nemaju.

Hocuspocus server obrađuje WebSocket konekcije i drži Yjs dokumente u memoriji. Kad se korisnik spoji, server kroz hook za autentifikaciju dohvati JWT i UUID plana i pita FastAPI server za ulogu korisnika u tom planu. Ako je korisnik čitatelj, konekcija se označava samo za čitanje i server odbacuje sve njegove dolazne izmjene. Kad se plan prvi put otvori, server učitava snimku iz baze, a svi sljedeći korisnici koji se spoje na isti plan dijele isti dokument u memoriji. Kad dva korisnika u isto vrijeme uređuju plan, Hocuspocus server primi obje izmjene i pošalje ih svim spojenim klijentima i svaki korisnik vidi promjenu drugog gotovo instantno.

Sadržaj plana također treba prenijeti iz Yjs dokumenta u relacijske tablice. Taj prijenos pokreće se nakon kratkog razdoblja neaktivnosti (30 sekundi nakon zadnje izmjene). Pri sinkronizaciji sadržaja plana s relacijskim tablicama, server čita trenutno stanje iz Yjs dokumenta i uspoređuje ga s onim što se već nalazi u relacijskim tablicama pa izračuna što je dodano, promijenjeno i obrisano. Redoslijed upisa je bitan jer neke tablice ovise o drugima pa se prvo upisuju stavke, a onda se brišu one kojih više nema. Osim toga, Yjs dokument se sprema natrag u bazu podataka kao snimka nakon zadnje izmjene (oko 2 sekunde) i pri odlasku zadnjeg korisnika ako je bilo promjena od zadnjeg spremanja, tako da se pri sljedećem otvaranju plana može vratiti posljednje stanje.

Proces suradnje u stvarnom vremenu može se vidjeti na Sl. 9.



Sl. 9. Proces suradnje u stvarnom vremenu

4.4. Rad bez mrežne veze

Aplikacije za putovanja se često koriste tamo gdje je mrežna veza loša (npr. u avionima, aerodromima ili hotelima). Ajmo rješava taj problem omogućujući rad aplikacije i bez mrežne veze.

Uz svaki Yjs dokument u pregledniku vežu se dva neovisna pružača usluge. IndexedDB pružatelj usluge pri svakoj izmjeni sprema dokument u lokalnu pohranu preglednika, a WebSocket pružatelj usluge drži vezu prema Hocuspocus serveru. Dokument je vezan uz UUID plana i postoji dok god korisnik ima plan otvoren, a WebSocket pružatelj usluge je vezan uz korisnikov token. Zbog IndexedDB, plan se učitava iz lokalne pohrane i prije nego što se WebSocket konekcija uopće uspostavi.

Redoslijed inicijalizacije pri otvaranju plana je bitan. WebSocket pružatelj usluge ne smije se spojiti dok IndexedDB pružatelj usluge ne završi s učitavanjem lokalnog stanja. Bez toga bi se prazan dokument spojio na server, primio trenutno stanje i tek onda pokušao dodati lokalne izmjene, što bi korisniku nakratko prikazalo prazan plan. To se može riješiti zastavicom u komponenti koja čeka da IndexedDB javi da je učitavanje gotovo prije nego se WebSocket pružatelj usluge uopće stvori.

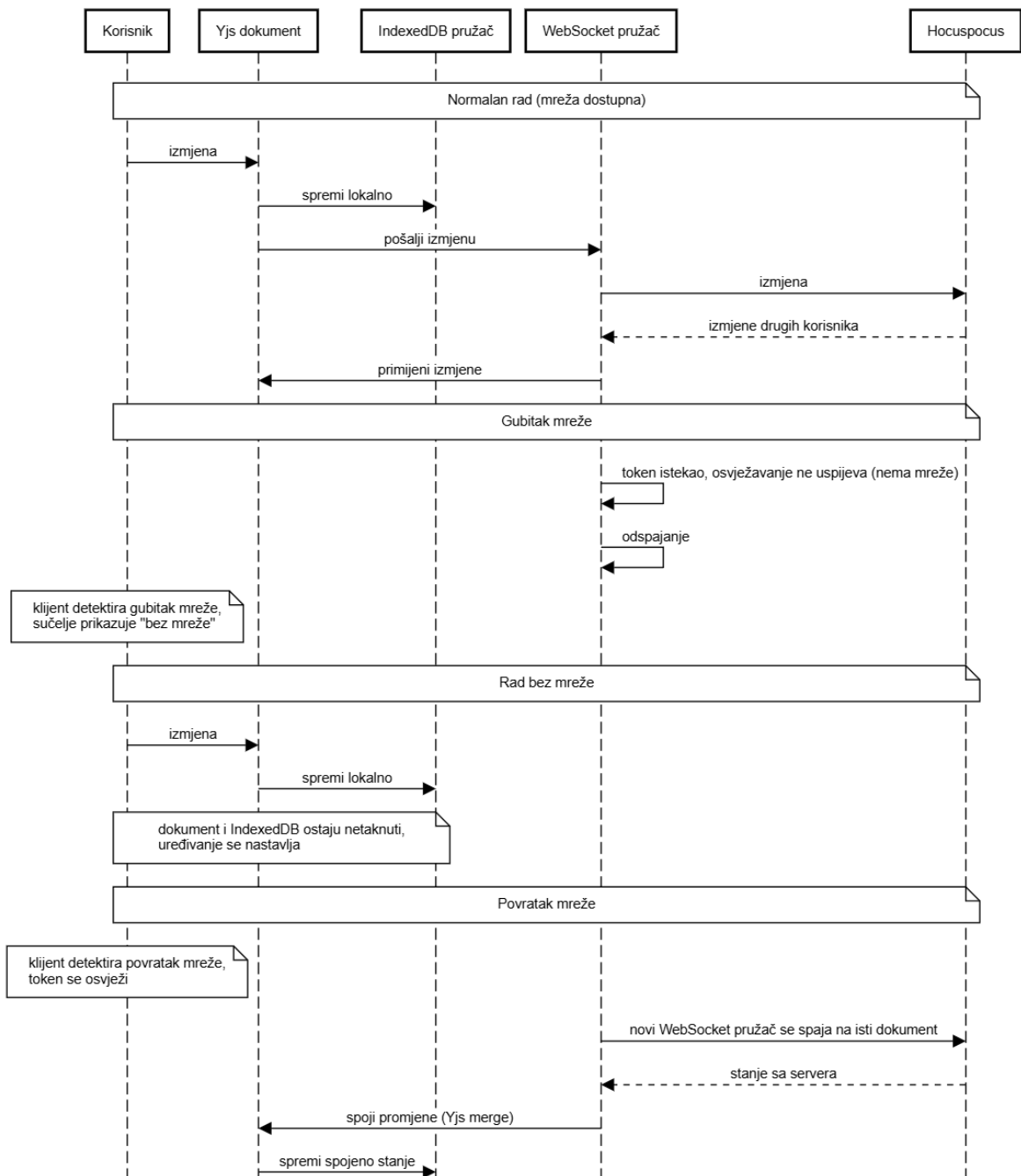
Prilikom prvog otvaranja plana, lokalna pohrana je prazna pa nema lokalnog stanja za učitati. Tada IndexedDB odmah javi da je gotov, a WebSocket pružatelj usluge se spoji i početno stanje stigne sa servera (neovisno o tome je li vlasnik već nešto unio ili je plan još prazan). Server učitava snimku plana iz baze, a ako snimka još ne postoji, rekonstruira dokument iz relacijskih tablica. Nakon toga lokalna kopija plana postoji u pregledniku.

Ako token istekne, a osvježavanje tokena ne uspije jer nema mrežne veze, WebSocket pružatelj usluge se odspoji. U tom slučaju dokument i njegova kopija u IndexedDB-u ostaju netaknuti, korisnik može nastaviti uređivati, a sučelje prikazuje da nema mrežne veze. Kada se mreža vrati i token osvježi, novi WebSocket pružatelj usluge se spoji na isti dokument i Yjs automatski sinkronizira sve izmjene napravljene u međuvremenu. Sl. 10 prikazuje životni ciklus veze, odnosno gubitak mreže i ponovno spajanje.

Pri otvaranju plana lokalna kopija može nakratko (ili dulje tijekom rada bez mrežne veze) biti zastarjela u odnosu na stanje na serveru, no to nije problem jer se pri ponovnom spajanju

lokalne i serverske izmjene spajaju, a ne prepisuju, pa svi korisnici na kraju dođu do istog stanja.

Pri odjavi, aplikacija briše lokalnu kopiju svih dokumenata. Bez toga bi sljedeći korisnik koji se prijavi na istom uređaju vidio planove prethodnog korisnika.



Sl. 10. Prikaz gubitka mreže i ponovno spajanje

4.5. AI obogaćivanje sadržaja

Kada korisnik doda mjesto u plan, sustav automatski dohvaća opis, radno vrijeme, cijenu, savjete, sliku, koordinate i polja specifična za tip stavke. Koristi se RAG pristup koji LLM-u daje svježije podatke s weba umjesto da se oslanja samo na ono što model zna iz treninga, što smanjuje vjerojatnost netočnih ili zastarjelih informacija.

Proces počinje normalizacijom korisničkog unosa. Naziv mjesta pretvara se u mali tekstualni ključ bez naglasaka i članaka, uz dodavanje destinacije kao kontekst. Taj ključ se uspoređuje s postojećim zapisima u bazi kako bi se različiti načini pisanja istog mjesta (npr. "Colosseum Rome", "the Colosseum" i "Colosseo") sveli na jedan zapis. Ako baza već ima podatke za to mjesto i rok trajanja nije istekao, odgovor se vrati odmah bez poziva prema vanjskim servisima.

Ako podaci nisu dostupni, Tavily pretražuje web i vraća nekoliko tekstova iz pouzdanih izvora, a ako pretraga ne vrati ništa, obogaćivanje se prekida. Ti tekstovi zajedno s korisničkim unosom se pošalju LLM-u, a paralelno se iz zasebnih servisa dohvaćaju slika i koordinate. Svi ovi podaci se upisuju u bazu tako što se fiksne informacije (npr. naziv, koordinate i opis) spremaju trajno, a promjenjive informacije (npr. radno vrijeme i cijene) s rokom trajanja od 24 sata.

Ako više korisnika u isto vrijeme traži obogaćivanje sadržaja za isto mjesto, sustav prepoznaje duplikat i ne pokreće novi poziv nego samo jedan i svi čekaju na isti rezultat.

Aplikacija podržava više LLM pružaca usluge (Gemini, Groq i lokalni Ollama), kojima pristupa izravno preko njihovih LangChain integracija, bez posredničkog servisa. LLM pružaci usluga su posloženi u lanac pa ako jedan dostigne kvotu, aplikacija prepoznaje grešku kvote i prebacuje se na sljedećeg iz liste. Validacijske i mrežne greške ne pokreću prebacivanje jer se odnose na problem s pozivom ili mrežom, a ne s pružačem usluge.

Postoji i *streaming* verzija krajnje točke koja vraća djelomične odgovore čim su generirani, što se koristi za prikaz AI prijedloga mjesta. Sustav korisniku predlaže nova mjesta na temelju njegovih preferencija i onoga što se već nalazi u planu.

Za prijevoz između destinacija ne koristi se LLM nego direktni pozivi prema Transitousu za javni prijevoz i OSRM-u za cestovne rute. LLM bi tu dodao samo latenciju i rizik od izmišljenih podataka.

4.5.1. Interakcije s LLM-om

Obogaćivanje sadržaja se izvršava kroz jedan poziv prema LLM-u kojemu se kroz LangChain prosljeđuje tekstualni *prompt*. *Prompt* LLM modelu zadaje ulogu pomoćnika koji izvlači činjenice o stavci određenog tipa, a kao ulaz mu se daje naziv mjesta koji je korisnik unio, destinaciju i rezultate Tavily pretrage. Ti se rezultati stave u prompt kao tekstualni kontekst pa model radi s njima umjesto s podacima na kojima je istreniran.

Pored toga, *prompt* zadaje i pravila da kanonski naziv mora opisivati isto mjesto koje je korisnik unio, polja za koja Tavily pretraga nije dala pouzdan odgovor moraju ostati prazna i model ne smije izmišljati podatke. Primjer jednog prompta nalazi se na Sl. 11.

```
You are a travel information assistant extracting facts about a hotel.

Name: Hotel Lutetia
Destination: Paris

Search results:
Hotel Lutetia, now Hôtel Lutetia, is a historic luxury hotel on the Left Bank of Paris, reopened in 2018 after renovation. It is operated by The Set Hotels.

Hôtel Lutetia Paris – Official Site: Located at 45 Boulevard Raspail, 75006 Paris. Art Deco landmark with spa, indoor pool, Akasha wellness center, and Saint-Germain brasserie. Check-in from 3:00 PM.

Hotel Lutetia Review: Rooms from around €900/night. Amenities include Michelin-level dining, a 700 sqm spa, fitness center, and 24h concierge. Booking recommended well in advance.

Extract: canonical_name, description, location (specific street address with number OR named area – never just the city), and these type-specific fields when known: amenities, check_in_time, price_range, booking_tips.
canonical_name MUST refer to the same hotel as the input Name above.
You may correct spelling or add an official brand prefix/suffix
(e.g. "Hilton" → "Hilton Paris Opera", "Hotel Lutetia" → "Mandarin Oriental Lutetia"), but never substitute a different establishment.
List fields must contain unique items only, no duplicates, at most 10 entries.
Set any field to null when the search results do not support a confident answer.
Never invent facts. Do not output an image URL – image lookup is handled outside the LLM.
```

Sl. 11. Primjer prompta za obogaćivanje sadržaja

Odgovor se traži u obliku strukturiranog izlaza, pa se natrag dobiva objekt s unaprijed određenim poljima. Tri su polja uvijek prisutna: kanonski naziv, opis i lokacija. Ostala polja ovise o tipu stavke i popunjavaju se kad postoje (npr. radno vrijeme, cijena i savjeti za atrakciju). Primjer odgovora nalazi se na Sl. 12. Slika i koordinate ne dolaze od LLM-a, nego se dohvaćaju s Pexelsa i Nominatima. Primjer odgovora nakon što su dohvaćeni podaci i s ostalih servisa nalazi se na Sl. 13.

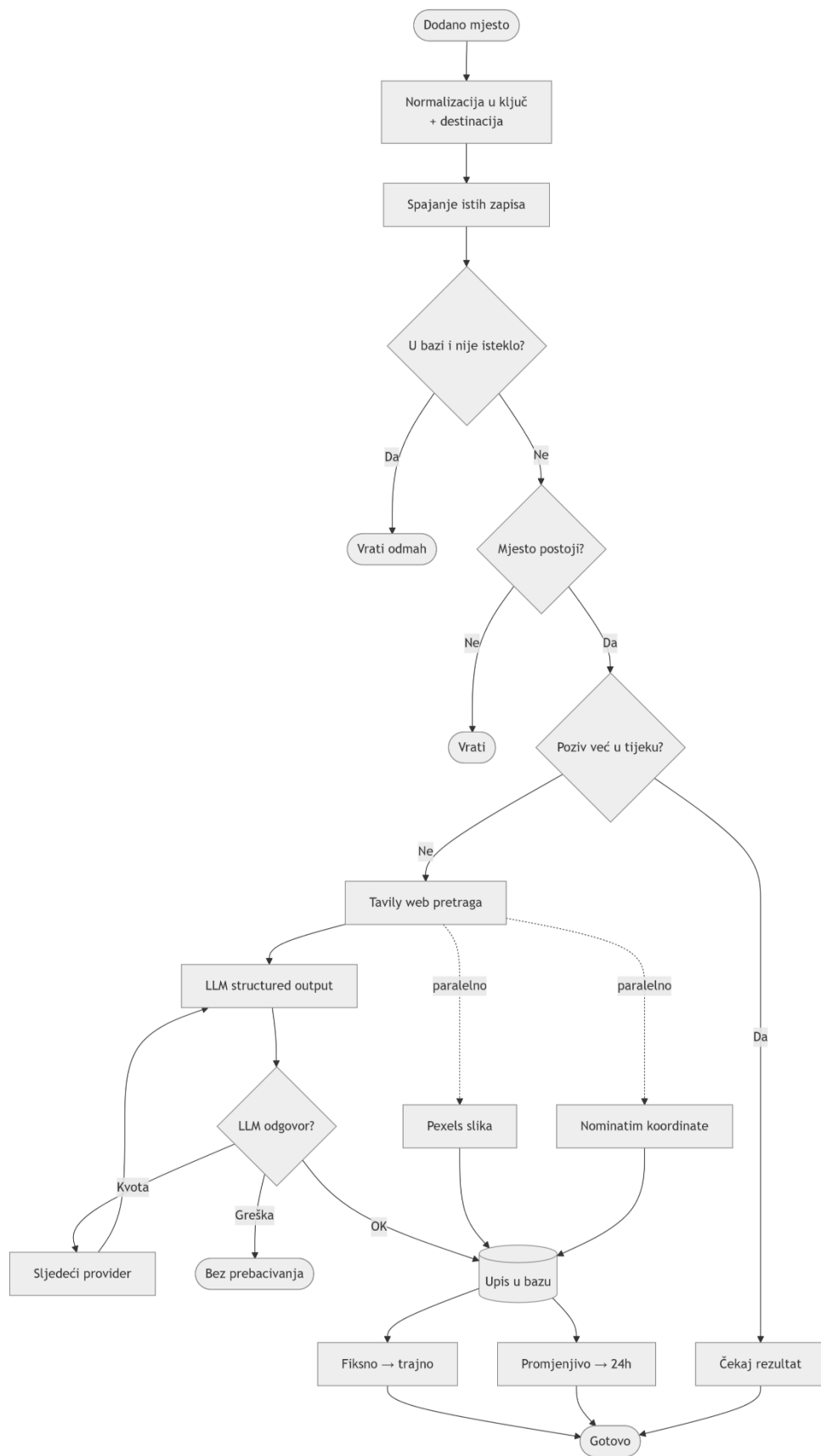
```
{
  "canonical_name": "Hôtel Lutetia",
  "description": "Historic Art Deco palace hotel on the Left Bank of Paris,
    reopened in 2018 after a major renovation. Known for its spa,
    fine dining, and Saint-Germain location.",
  "location": "45 Boulevard Raspail, 75006 Paris",
  "amenities": ["Spa", "Indoor pool", "Akasha wellness center", "Michelin-level dining",
    "Fitness center", "24h concierge"],
  "check_in_time": "3:00 PM",
  "price_range": "€€€€",
  "booking_tips": "Book well in advance, especially for peak season and spa packages.",
  "opening_hours": null,
  "tips": null,
  "cuisine": null,
  "reservation_tips": null,
  "schedule": null,
  "duration": null,
  "categories": ["hotel", "luxury", "historic"]
}
```

Sl. 12. Odgovor LLM-a

```
{
  "place_id": "a1b2c3d4-...",
  "canonical_name": "Hôtel Lutetia",
  "description": "Historic Art Deco palace hotel on the Left Bank of Paris,
    reopened in 2018 after a major renovation. Known for its spa,
    fine dining, and Saint-Germain location.",
  "location": "45 Boulevard Raspail, 75006 Paris",
  "image_url": "https://images.pexels.com/photos/.../lutetia.jpg",
  "lat": 48.8517,
  "lng": 2.3265,
  "timezone": "Europe/Paris",
  "amenities": ["Spa", "Indoor pool", "Akasha wellness center", "Michelin-level dining",
    "Fitness center", "24h concierge"],
  "check_in_time": "3:00 PM",
  "price_range": "€€€€",
  "booking_tips": "Book well in advance, especially for peak season and spa packages.",
  "opening_hours": null,
  "tips": null,
  "cuisine": null,
  "reservation_tips": null,
  "schedule": null,
  "duration": null,
  "categories": ["hotel", "luxury", "historic"]
}
```

Sl. 13. Konačni odgovor

Proces AI obogaćivanja sadržaja se može vidjeti na Sl. 14.



Sl. 14. Proces AI obogaćivanja sadržaja

4.6. Sigurnost

Korisnički identitet vodi Supabase Auth koji pokriva registraciju putem e-maila i lozinke te Google OAuth, potvrdu e-maila, oporavak lozinke i izdavanje JWT tokena. Svaki korisnik ima jedan zapis u tablici korisnika i profil koji se automatski stvara okidačem u bazi pri registraciji.

JWT token kratko traje i Supabase SDK ga automatski osvježava iz *refresh* tokena pohranjenog u HTTP-only kolačiću. *Middleware* na svakom zahtjevu verificira token i prosljeđuje zahtjev ili preusmjerava na stranicu za prijavu. Isti token koriste sva tri dijela sustava: Next.js ga čita iz kolačića, FastAPI iz Authorization zaglavlja, a Hocuspocus server iz WebSocket Authorization zaglavlja pri uspostavi mrežne veze.

Autorizacija u bazi radi kroz RLS pravila koja osiguravaju da korisnik može čitati i pisati samo ono na što ima pravo. Čitanje sadržaja plana zahtijeva da korisnik bude član plana, pisanje zahtijeva ulogu uređivača, a operacije na razini plana zahtijevaju ulogu vlasništvo. Vidljivost planova provjerava se kombinacijom članstva, razine vidljivosti i prijateljstva. Tablice kojima pristupa samo poslužitelj nemaju RLS pravila jer im se pristupa preko *service role* ključa koji ta pravila ionako zaobilazi. Te tablice nisu izložene pregledniku jer im frontend ne pristupa izravno, nego isključivo kroz FastAPI backend.

Provjera korisničke uloge unutar plana radi se na tri neovisna sloja. Sučelje ne prikazuje kontrole koje korisnik nema pravo koristiti. Hocuspocus server označava konekciju samo za čitanje i odbacuje sve dolazne poruke korisnika koji nemaju pravo uređivanja. Baza odbija neovlaštene upise kroz RLS pravila. Svaki od tih slojeva radi neovisno, što znači da zaobilazanje jednog ne daje pristup kroz ostale.

Komunikacija između Hocuspocus servera i FastAPI servera zaštićena je dijeljenom tajnom u zaglavlju svakog zahtjeva. Tajna se uspoređuje na način koji sprječava vremenske napade, a zahtjevi bez ispravnog zaglavlja se odmah odbijaju.

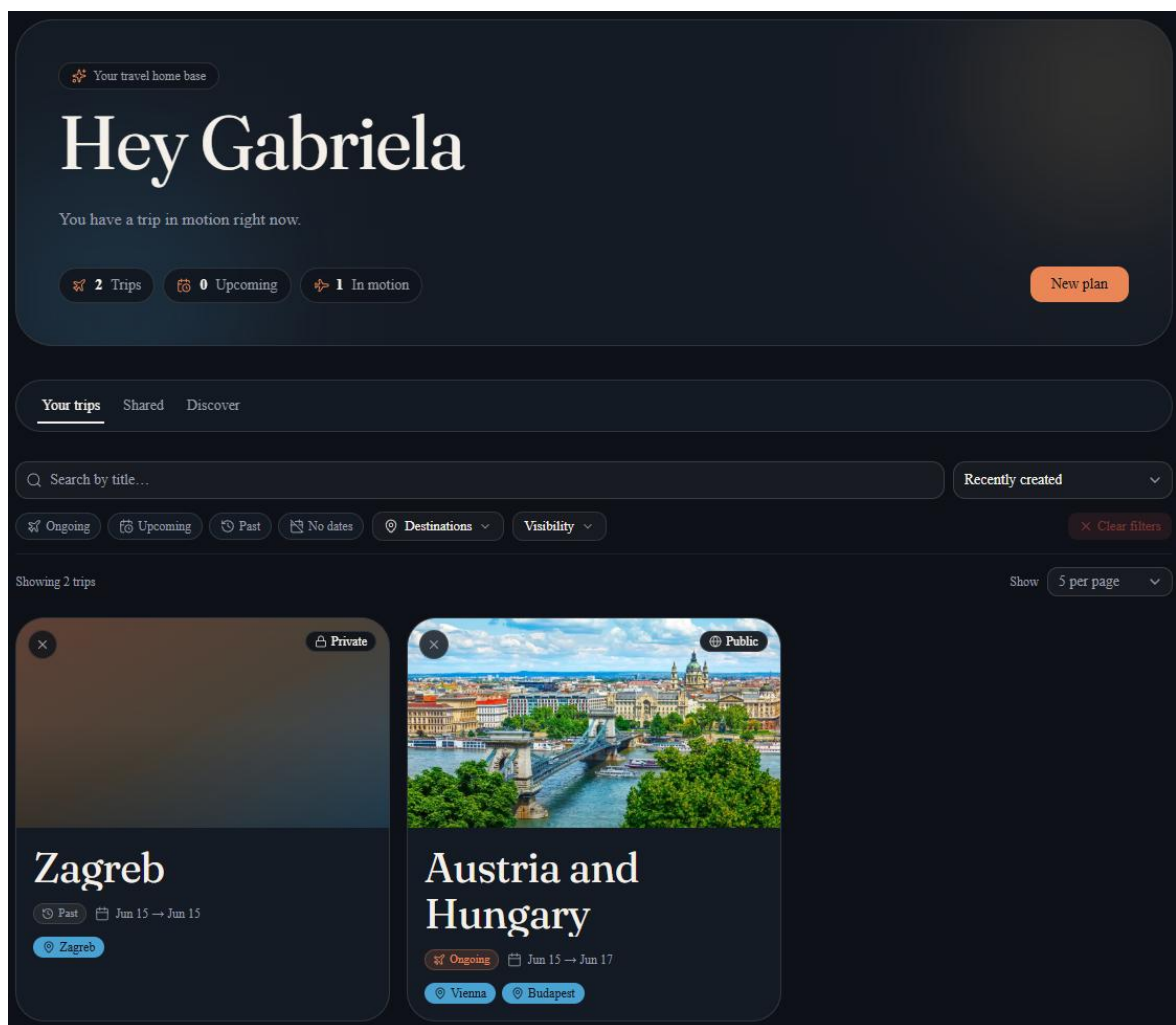
Svi API ključevi, uključujući *service role* ključ i ključeve LLM pružača usluge, dostupni su samo na serveru kroz *environment* varijable. Preglednik ih nikad ne dobiva.

4.7. Stranice i komponente

Sučelje se sastoji od nekoliko stranica izrađenih od React komponenti, a organizirane su po funkcionalnosti.

Početna stranica

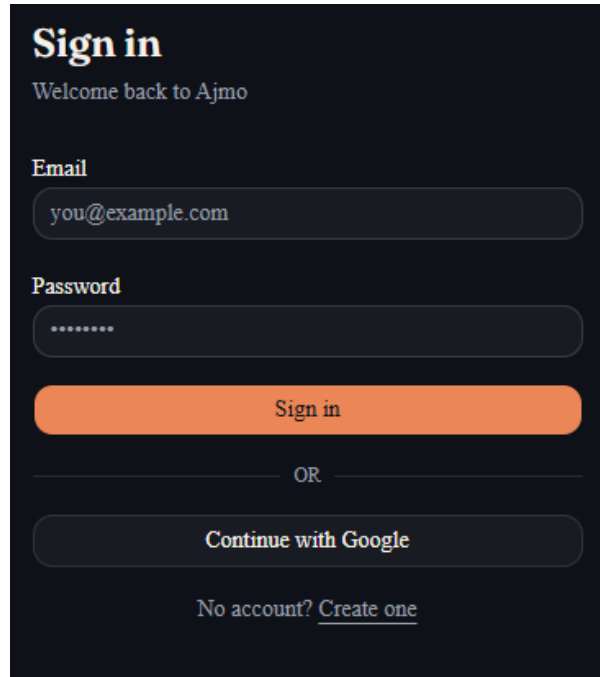
Početna stranica je glavna stranica za prijavljene korisnike. Na vrhu prikazuje pozdravnu poruku s korisničkim imenom i kratke statistike (broj planova, nadolazećih i aktivnih putovanja). Odavde se može otvoriti *wizard* za kreiranje novog plana. Ispod su tri kartice: "Your trips" s korisnikovim planovima, "Shared" s planovima na koje je pozvan i "Discover" s javnim planovima ostalih korisnika. Svaka kartica ima pretraživanje po naslovu, filtre po statusu (Ongoing, Upcoming, Past) i destinaciji te sortiranje. Planovi se prikazuju kao kartice s naslovnom slikom, nazivom, datumima i destinacijama. Početna stranica se može vidjeti na Sl. 15.



Sl. 15. Početna stranica

Prijava i registracija

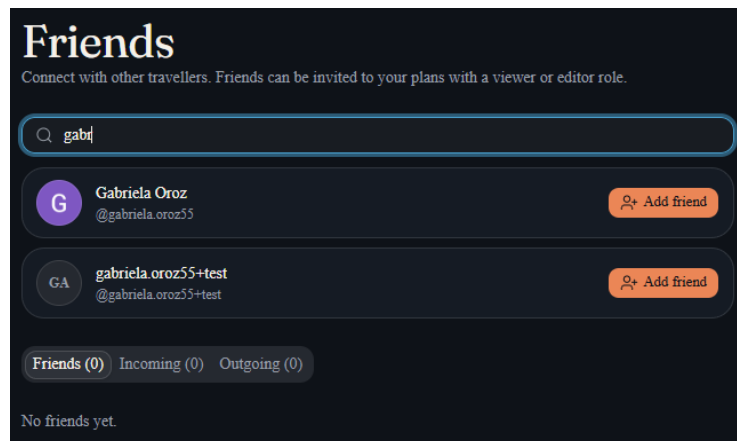
Prijava i registracija su dvije odvojene stranice, ali obje nude prijavu/registraciju e-mailom i lozinkom ili Google računom. Nakon uspješne registracije Supabase Auth šalje potvrdni e-mail, a tek nakon potvrde e-maila korisnik može koristiti aplikaciju. Stranica za prijavu se može vidjeti na Sl. 16.



Sl. 16. Stranica za prijavu

Prijatelji

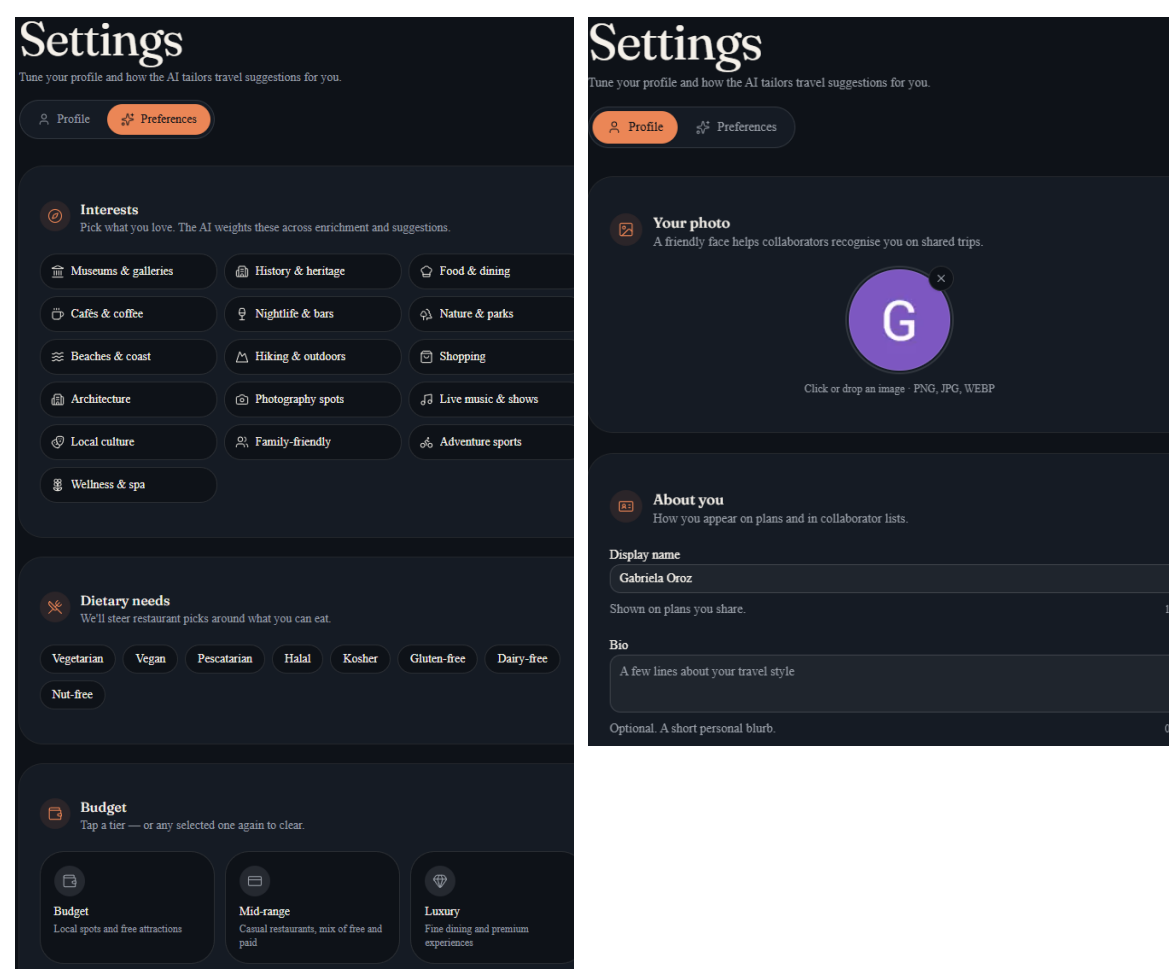
Stranica za prijatelje omogućuje pretraživanje korisnika po imenu ili korisničkom imenu. Rezultati se prikazuju odmah ispod polja za pretraživanje s opcijom slanja zahtjeva za prijateljstvo. Ispod su tri kartice: „Friends“ s prihvaćenim prijateljima, „Incoming“ s primljenim zahtjevima i „Outgoing“ s poslanim zahtjevima. Ova stranica se može vidjeti na Sl. 17.



Sl. 17. Stranica za prijateljstva

Postavke

Stranica s dvije kartice. U kartici „Profile“ korisnik može promijeniti profilnu sliku, ime koje se prikazuje i biografiju. U kartici „Preferences“ može odabrati interese (npr. *Museums*, *Nightlife*, *Hiking*), prehranu (npr. *Vegan*, *Halal*, *Gluten-free*) i budžet (*Budget*, *Mid-range*, *Luxury*). Te preferencije LLM koristi pri generiranju AI prijedloga. Stranica za postavke se može vidjeti na Sl. 18.



Sl. 18. Postavke

Kreiranje plana

Novi plan se kreira kroz *wizard* s četiri koraka. U prvom koraku korisnik unosi naslov, datume i opis. U drugom dodaje destinacije (par država i grad) i može im odmah dodijeliti dane. U trećem postavlja naslovnu sliku. Četvrti korak je pregled svega prije kreiranja. Ovo se može vidjeti na Sl. 19.

The image displays two sequential screenshots of a travel plan creation wizard. The first screenshot, titled "Where are we going?", shows a form with a progress bar at the top indicating four steps: 1. Trip, 2. Destinations, 3. Cover, and 4. Review. The form includes a "Title" field with the text "USA & Canada", "From" and "To" date pickers set to "24/05/2026" and "05/06/2026" respectively, and a "Description (optional)" text area. The second screenshot, titled "Pick your destinations", shows the same progress bar with the second step, "Destinations", highlighted. It features two destination cards. The first card, for "Canada" and "Toronto", lists 13 days with checkboxes; days 1 through 13 are all checked. The second card, for "USA" and "New York City", also lists 13 days; days 3, 4, 5, 8, and 9 are checked, while days 1, 2, 6, 7, 10, 11, 12, and 13 are unchecked. Both screenshots have a "Back" button and a "Next" button at the bottom.

Sl. 19. Prikaz kreiranja plana

Uređivač putovanja

Glavna i najsloženija stranica je uređivač putovanja. Na vrhu je zaglavlje s nazivom plana, datumima, destinacijama i značkom statusa konekcije (sinkronizirano, spremanje, ponovno spajanje ili bez mreže), a ispod zaglavlja su AI prijedlozi s gumbom za osvježavanje i kartice za navigaciju između dana. Lijevo se nalazi sadržaj, a desno karta na kojoj se vide oznake za sve stavke (trenutnog dana ili svih dana zajedno).

Za svaki dan prikazuju se hoteli, dnevne bilješke, polje za unos za dodavanje novih stavki i lista stavki. Kod dodavanja novih stavki korisnik odabere njezin tip (atrakcija, restoran ili aktivnost) i upiše naziv mjesta, a sustav automatski dohvaća podatke o tom mjestu (opis, lokaciju, sliku, radno vrijeme, cijenu i slično) postupkom AI obogaćivanja koje je opisano

u poglavlju 4.5. Stavka se zatim dodaje na listu i prikazuje sa slikom, lokacijom, cijenom i mogućnošću ocjenjivanja i odabira oznake sviđanja.

Između stavki se prikazuju prijedlozi za usmjeravanje (pješke, bicikl, auto, javni prijevoz) s trajanjem i udaljenošću. Međugradski transport prikazuje se kao *Cross-city* stavka unutar dana.

Navedeno se može vidjeti na Sl. 20.

ajmo

Plans6



USA & Canada

May 24 → Jun 5, 2026PrivateToronto, Canada · New York City, USA · Philadelphia, USA · Washington DC, USA

ShareChatActivitySettings

AI SUGGESTIONS

CN Tower

Toronto

Panoramic city views

~\$30

+ Add

Carbone

New York City

Retro Italian-American cuisine

€6€

+ Add

Independence Hall guided tour

Philadelphia

Historic US landmarks

~\$10

+ Add

National Gallery of Art

Washington DC

World-class art museum

Free

+ Add

Central Park bike tour

New York City

Scenic park ride

~\$20

+ Add

Day 12026-05-24

Day 22026-05-25

Day 32026-05-26

Day 42026-05-27

Day 52026-05-28

Day 62026-05-29

Day 72026-05-30

Day 82026-05-31

Day 92026-06-01

Day 102026-06-02

Day 112026-06-03

Day 122026-06-04

Day 132026-06-05

Day 62026-05-29

Hotel Stanford

STAY · DAYS 3-6

Midtown Manhattan

Check-out

Hotel check-in 4:00 PM

Washington Plaza

STAY · DAYS 6-7

10 Thomas Circle NW, ...

Check-in

Hotel check-in 3:00 PM

\$ 105

DAY NOTES

What's the plan for this day? Any notes...

Bus

CROSS-CITY

3 h 23 min160 kmBus M1 + Coach Greyhound US0670 + Bus 32 + walk

PHILADELPHIA, USA

Attraction

Restaurant

Activity

e.g. Eiffel Tower

Add note

rocky steps

Attraction

2600 Benjamin Franklin Parkway, Phila...

\$ 50-\$140

Bike

15 min2.5 km

Philadelphia City Hall

Attraction

1450 John F Kennedy Blvd, Philadelphi...

\$ 8-\$15

Bus

CROSS-CITY

4 h 12 min232 kmCoach Greyhound US0320 + Coach Greyhound US0310 + Bus D20 + walk

Cross-city transport

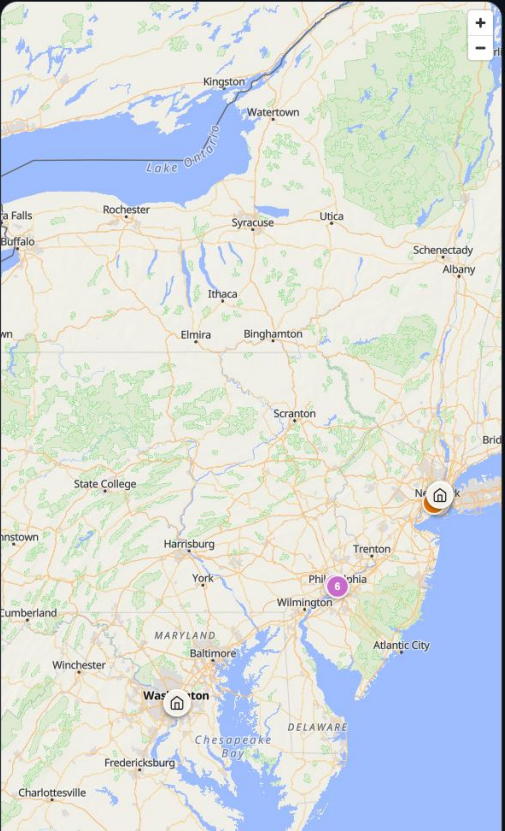
Plan how you'll get between cities

All days

Active day only

Cities 4/4

Fit



Sl. 20. Uređivač putovanja

5. Budući rad

Sustav se može proširiti i poboljšati na nekoliko načina bez većih promjena u arhitekturi.

Trenutno AI prijedlozi predlažu jedno po jedno mjesto, što može biti sporo za korisnike koji tek počinju planirati putovanje. Bilo bi korisno da sustav na temelju destinacije i datuma može predložiti cijeli plan ili itinerar odjednom, a korisnik bi ga onda samo prilagodio prema vlastitim željama.

Sučelje je trenutno dostupno samo na engleskom jeziku, što ograničava korisnike, pa bi bilo dobro dodati podršku za više jezika.

Jedan od većih problema trenutne implementacije su podaci o lokacijama. Ni vanjski API servisi ni LLM ne daju uvijek točne informacije, posebno kad je riječ o koordinatama i usmjeravanju. Ovo je područje koje bi svakako trebalo poboljšati, ali nije jednostavno pronaći rješenje koje je i pouzdano i besplatno.

Trenutna arhitektura s tri odvojena procesa komplicira razvoj i održavanje. Python je potreban zbog AI dijela, a Node.js zbog Yjs WebSocket servera, pa ih nije moguće jednostavno spojiti u jedan proces. Jedan od mogućih smjerova bi bio prelazak na mikroservisnu arhitekturu, ali to bi zahtijevalo detaljniju analizu i dosta refaktoriranja.

Što se tiče proširivanja, izrada mobilne aplikacije bi mogla biti sljedeći korak ovog projekta, budući da Yjs već radi na mobilnim uređajima i dizajn bez mrežne veze se može prenijeti bez puno dodatnog posla.

Zaključak

Cilj ovog rada bio je napraviti web aplikaciju za zajedničko planiranje putovanja koja u jedinstvenoj platformi spaja uređivanje itinerara u stvarnom vremenu, rad bez mrežne veze i AI obogaćivanje sadržaja. Nijedna postojeća aplikacija ne pokriva sve tri stvari istovremeno, što je bio glavni razlog za razvoj aplikacije Ajmo.

Za zajedničko uređivanje koristi se Yjs CRDT dokument kojim upravlja Hocuspocus WebSocket server. Kad svi korisnici zatvore plan i prođe kratko vrijeme neaktivnosti, server pročita trenutno stanje dokumenta i spremi ga u relacijske tablice. Za rad bez mrežne veze, uz svaki Yjs dokument vezan je IndexedDB pružatelj usluge koji pri svakoj promjeni sprema dokument u lokalnu pohranu preglednika, pa korisnik može nastaviti uređivati i bez interneta, a kad se mreža vrati Yjs automatski sinkronizira sve izmjene. Kada korisnik želi dodati mjesto u plan, sustav automatski dohvaća informacije (npr. opis, radno vrijeme, cijenu, savjete i sliku). Za ovo se koristi RAG pristup koji LLM-u daje svježije podatke s weba, što smanjuje vjerojatnost netočnih ili zastarjelih informacija.

Najveći otvoreni problem ostaju podaci o lokacijama, gdje ni vanjski API servisi ni LLM ne daju uvijek točne rezultate. To je područje koje bi u budućnosti trebalo poboljšati.

Literatura

- [1] »Wanderlog,« 2022. [Mrežno]. Available: <https://wanderlog.com/>.
- [2] »Tripit,« 2026. [Mrežno]. Available: <https://www.tripit.com/web>.
- [3] »Roadtrippers,« 2026. [Mrežno]. Available: <https://roadtrippers.com/>.
- [4] »Google Travel,« [Mrežno]. Available: <https://www.google.com/travel/explore>.
- [5] »Yjs Docs,« 2026. [Mrežno]. Available: <https://docs.yjs.dev/>.
- [6] »Next.js Docs,« 2026. [Mrežno]. Available: <https://nextjs.org/docs>.
- [7] »React,« 2026. [Mrežno]. Available: <https://react.dev/>.
- [8] »TypeScript is JavaScript with syntax for types,« 2026. [Mrežno]. Available: <https://www.typescriptlang.org/>.
- [9] »Hocuspocus,« 2025. [Mrežno]. Available: <https://tiptap.dev/docs/hocuspocus/getting-started/overview>.
- [10] S. Ramírez, »FastAPI,« 2018. [Mrežno]. Available: <https://devdocs.io/fastapi/>.
- [11] »Langchain - Structured output,« [Mrežno]. Available: <https://docs.langchain.com/oss/python/langchain/structured-output>.
- [12] »Langchain - Streaming,« [Mrežno]. Available: <https://docs.langchain.com/oss/python/langchain/streaming>.
- [13] »Supabase Docs,« [Mrežno]. Available: <https://supabase.com/docs>.
- [14] »Tavily,« [Mrežno]. Available: <https://docs.tavily.com/documentation/about>.
- [15] »Pexels API,« 2026. [Mrežno]. Available: <https://publicapi.dev/pexels-api>.
- [16] »Nominatim,« [Mrežno]. Available: <https://nominatim.org/>.
- [17] »Photon REST API,« [Mrežno]. Available: https://photonnetwork.readthedocs.io/en/v1.0.0/rest_api/.
- [18] »Transitous,« 2026. [Mrežno]. Available: <https://transitous.org/api/>.
- [19] D. Tasevski, »Building Collaborative Interfaces: Operational Transforms vs. CRDTs,« 8 8 2025. [Mrežno]. Available: <https://dev.to/puritanic/building-collaborative-interfaces-operational-transforms-vs-crdts-2obo>.
- [20] »Conflict-free replicated data type,« 20 6 2026. [Mrežno]. Available: https://en.wikipedia.org/wiki/Conflict-free_replicated_data_type.

Izjava o korištenju umjetne inteligencije

Tijekom izrade rada korišten je Claude, modeli Opus i Sonnet, putem alata Claude Code i web stranice claude.ai za generiranje, refaktoriranje i otklanjanje pogrešaka izvornog koda aplikacije, za prijedloge i raspravu o odlukama, izradu dijagrama i procesnih shema u radu te jezično i stilsko dotjerivanje teksta.

Osvrt na korištenje umjetne inteligencije: Umjetnu inteligenciju koristila sam kao pomoćni alat tijekom cijelog procesa, no vodstvo nad radom i sve ključne odluke su bile moje. Definirala sam cilj i zahtjeve sustava te osmislila arhitekturu i podatkovni model. Svaki prijedlog i generirani kod sam proučila, ručno testirala, prilagodila, doradila i integrirala u cjelinu. Tijekom korištenja alata umjetne inteligencije primjetila sam da model ponekad predlaže nepostojeće metode za neke biblioteke.

Izjava o korištenju umjetne inteligencije: Potvrđujem da sam tijekom izrade ovog diplomskog rada koristila umjetnu inteligenciju u skladu s Politikom primjerenog korištenja umjetne inteligencije na Fakultetu elektrotehnike i računarstva, te da umjetna inteligencija nije zamijenila moje kritičko razmišljanje niti moj doprinos.